

INTEGRA · CONTROL DE CLIENTES

Reporte técnico de implementación

Arquitectura, tecnologías, modelo de datos, seguridad, calidad y despliegue

Versión 3.0 (migración 0003)

Octubre de 2026

Dirigido a: personal técnico y responsables del sistema

Contenido

1. Resumen ejecutivo.....	3
1.1 Alcance funcional	3
2. Arquitectura	4
2.1 Capas del backend	5
2.2 Estructura del frontend.....	5
3. Tecnologías utilizadas	6
3.1 Backend.....	6
3.2 Frontend.....	6
3.3 Base de datos e infraestructura	7
4. Modelo de datos	8
4.1 Diagrama entidad-relación	8
4.2 Descripción de las tablas.....	9
4.3 Integridad garantizada por la base de datos	9
4.4 Migraciones.....	10
5. Reglas de negocio implementadas	11
5.1 Comisiones y saldos	11
5.2 Acceso a la información	11
5.3 Cambio de empresa reversible	11
6. API REST	12
7. Seguridad	13
8. Calidad: pruebas e integración continua	14
8.1 Pruebas automáticas.....	14
8.2 Integración continua (GitHub Actions)	14
9. Despliegue y operación.....	16
9.1 Producción (Render)	16
9.2 Variables de entorno.....	16
9.3 Entorno local de desarrollo.....	17
10. Limitaciones y trabajo futuro.....	18

1. Resumen ejecutivo

Este documento describe la implementación del sistema **Integra · Control de Clientes**: una aplicación web para controlar las entradas (con comisión por proyecto), las salidas y los saldos de los clientes de varias empresas, con reportes exportables y bitácora de auditoría.

Aspecto	Resumen
Tipo de sistema	Aplicación web de una sola página (SPA) con API REST.
Backend	Python 3.13 · FastAPI · SQLAlchemy 2 · Alembic · PostgreSQL 16.
Frontend	React 19 · TypeScript · Vite 8 · Tailwind CSS 4 · TanStack Query 5.
Seguridad	JWT (HS256), contraseñas con bcrypt, límite de intentos de acceso, control de acceso por responsable.
Calidad	41 pruebas automáticas, verificación de migraciones, integración continua en GitHub Actions y Dependabot.
Despliegue	Imagen Docker única (API + frontend) en Render, con PostgreSQL administrado; despliegue automático solo si pasan las pruebas.
Repositorio	github.com/carlosfloresmartinez1993/api_bancaria (privado), rama main.

1.1 Alcance funcional

- Usuarios con roles **administrador** y **contador**; cada contador ve solo lo suyo.
- Jerarquía **Empresa → Cliente → Proyecto**; los clientes pueden existir **sin empresa** y cambiar de empresa de forma **reversible**.
- Comisión por proyecto con **historial de vigencias** y recálculo automático de entradas.
- Entradas con método de pago y marca de factura; salidas por proyecto; saldos calculados al momento.
- Constructor de reportes y reportes fijos, exportables a Excel, PDF y CSV con «Elaborado por».
- Panel de inicio con totales, saldo por empresa y **avance del día** por trabajador.
- Bitácora de auditoría inmutable y cierre diario por correo (opcional).

2. Arquitectura

El sistema sigue una arquitectura de **tres capas** desplegada como un único contenedor: el navegador ejecuta la SPA de React, que se comunica por HTTPS/JSON con la API FastAPI; la API delega la lógica en servicios de negocio que acceden a PostgreSQL mediante SQLAlchemy.

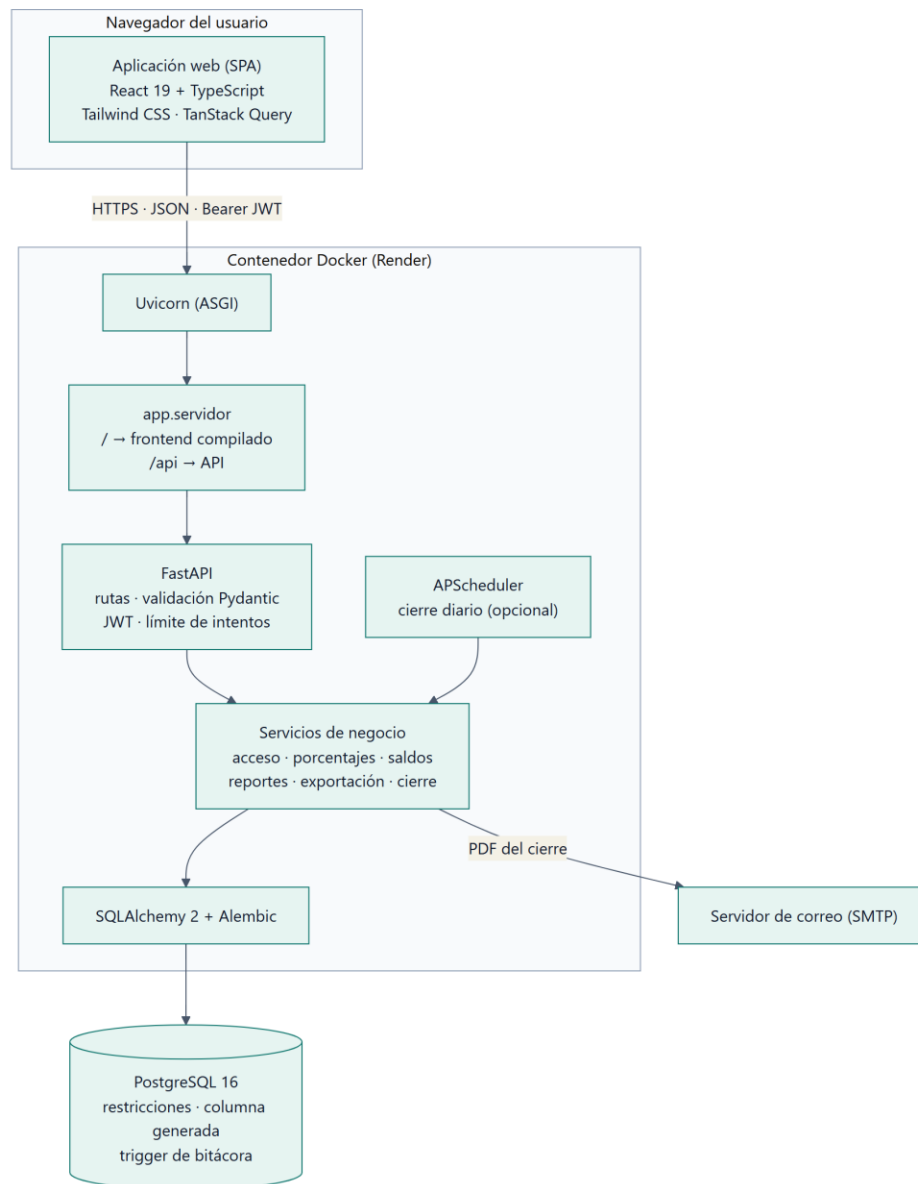


Figura 1. Arquitectura lógica del sistema.

2.1 Capas del backend

Capa	Ubicación	Responsabilidad
Rutas (API)	app/api/routes/	Endpoints REST, validación de entrada/salida con Pydantic, dependencias de autenticación.
Esquemas	app/schemas/	Modelos Pydantic v2 de petición y respuesta.
Servicios	app/services/	Reglas de negocio: acceso, porcentajes, saldos, reportes, exportación, métodos de pago, cierre.
Modelos	app/models/	Entidades SQLAlchemy 2 (tipado Mapped) y restricciones.
Núcleo	app/core/	Configuración (pydantic-settings), seguridad (JWT, bcrypt), errores, límite de intentos, zona horaria.
Base de datos	app/db/, alembic/	Sesión, convención de nombres y migraciones versionadas.
Tareas	app/jobs/	Cierre diario programado con APScheduler.
Servidor	app/servidor.py	Monta la API en /api y sirve el frontend compilado en /.

2.2 Estructura del frontend

Carpeta	Contenido
src/pages/	Pantallas: Inicio, Empresas, EmpresaDetalle, Clientes, ClienteDetalle, Entradas, Salidas, Reportes, MétodosPago, Usuarios, Perfil, Login.
src/components/	Layout, selectores encadenados (empresa → cliente → proyecto), formularios, AvanceDelDia, componentes de interfaz.
src/lib/	Cliente HTTP (api.ts), tipos, consultas TanStack Query, filtros en la URL, invalidación de caché, utilidades de empresa.

3. Tecnologías utilizadas

Las versiones del backend están **fijadas** en `requirements.txt` (generado con uv a partir de `requirements.in`) y las del frontend en `package-lock.json`; así cada instalación es idéntica.

3.1 Backend

Tecnología	Versión	Uso
Python	3.13	Lenguaje del backend.
FastAPI	0.142.1	Framework web de la API REST y documentación OpenAPI.
Uvicorn	0.54.0	Servidor ASGI.
SQLAlchemy	2.1.1	ORM y construcción de consultas de reportes.
Alembic	1.20.0	Migraciones de esquema.
psycopg	3.3.6	Controlador de PostgreSQL.
Pydantic / pydantic-settings	2.13.5 / 2.15.0	Validación de datos y configuración por variables de entorno.
PyJWT	2.15.1	Emisión y verificación de tokens JWT.
bcrypt	5.0.0	Hash de contraseñas.
slowapi	0.1.10	Límite de intentos de inicio de sesión.
APScheduler	3.11.3	Cierre diario programado.
openpyxl	3.1.5	Exportación a Excel.
reportlab	5.0.1	Exportación a PDF.
pytest / httpx	9.1.1 / 0.28.1	Pruebas automáticas de la API.

3.2 Frontend

Tecnología	Versión	Uso
React / React DOM	19.3	Biblioteca de interfaz.
TypeScript	7.0	Tipado estático.
Vite	8.3	Servidor de desarrollo (con proxy /api) y compilación.
Tailwind CSS	4.3	Estilos utilitarios.

Tecnología	Versión	Uso
React Router	8.4	Navegación entre pantallas.
TanStack Query	5.104	Consultas al servidor, caché e invalidación.
Node.js	22	Compilación del frontend.

3.3 Base de datos e infraestructura

Tecnología	Uso
PostgreSQL 16	Base de datos relacional; columnas generadas, restricciones CHECK, índices únicos parciales y triggers.
Docker	Imagen única en dos etapas: <code>node:22-slim</code> compila el frontend y <code>python:3.13-slim</code> ejecuta la API (usuario sin privilegios).
Render	Servicio web (plan gratuito de prueba) y PostgreSQL administrado, definidos en <code>render.yaml</code> (Blueprint).
GitHub	Repositorio privado y control de versiones.
GitHub Actions	Integración continua: migraciones, pruebas, compilación del frontend y de la imagen Docker.
Dependabot	Propuestas de actualización: pip y npm cada semana; imágenes Docker y GitHub Actions cada mes.
uv	Compilación de las dependencias de Python con versiones exactas.

4. Modelo de datos

4.1 Diagrama entidad-relación

El diagrama muestra las nueve tablas del sistema, sus columnas principales y sus relaciones. Las llaves primarias son UUID.

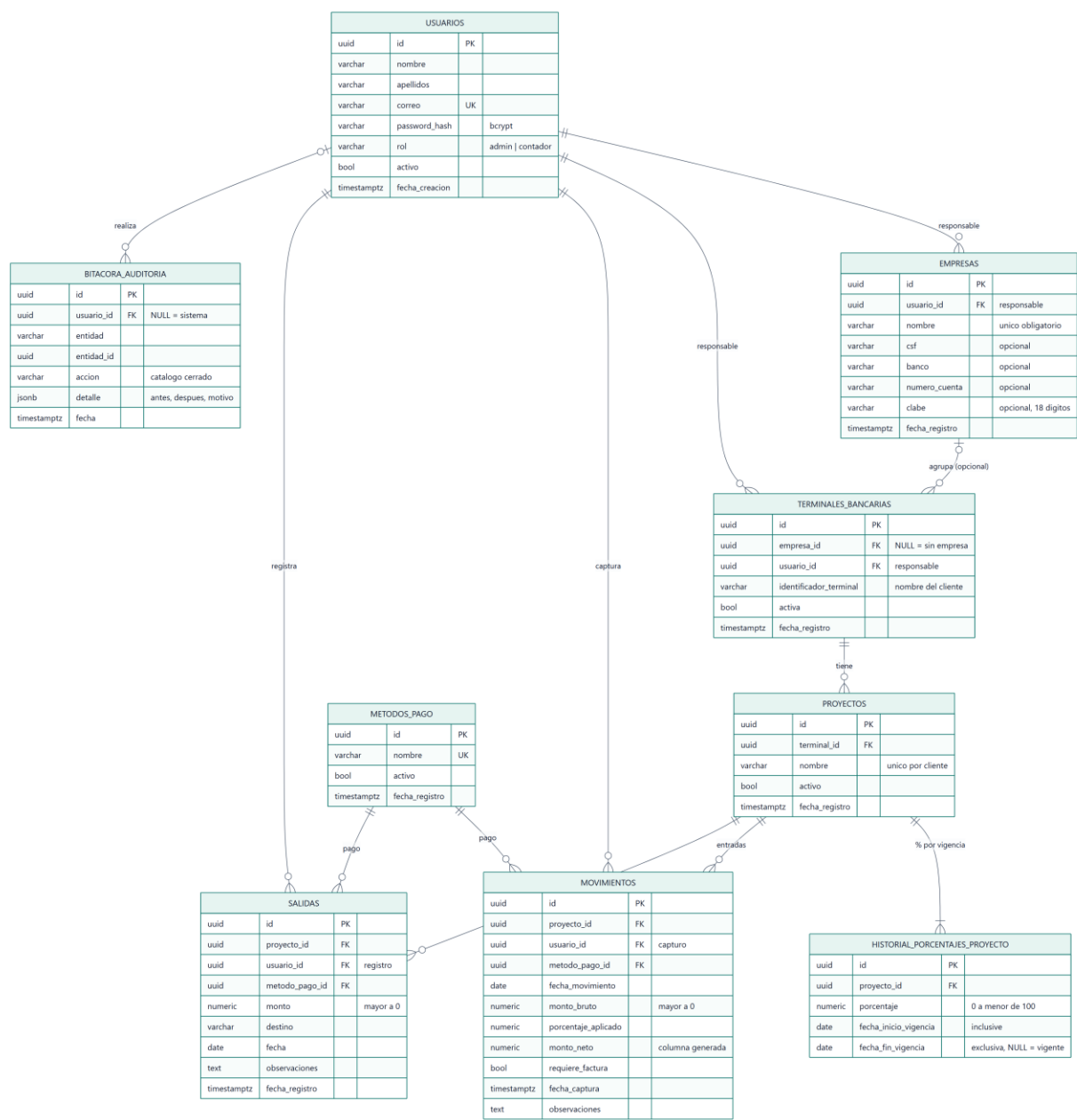


Figura 2. Diagrama entidad-relación.

4.2 Descripción de las tablas

Tabla	Descripción
usuarios	Administradores y contadores. Correo único, contraseña con bcrypt, rol y estado activo.
empresas	Agrupar clientes. Solo el nombre es obligatorio (único). usuario_id es el contador responsable.
terminales_bancarias	Los clientes . empresa_id es opcional (NULL = sin empresa); usuario_id es el responsable.
proyectos	Proyectos de cada cliente; nombre único por cliente; activo/inactivo.
historial_porcentajes_proyecto	Comisión del proyecto por periodo de vigencia [inicio, fin).
metodos_pago	Catálogo: Transferencia, Depósito, Efectivo y los que se agreguen.
movimientos	Entradas: monto bruto, % aplicado, neto (generado), método de pago, factura, quién capturó.
salidas	Salidas por proyecto: monto, destino, fecha, método de pago, quién registró.
bitacora_auditoria	Registro inmutable de acciones con detalle JSONB (antes, después, motivo).

4.3 Integridad garantizada por la base de datos

Las reglas críticas no dependen solo del código: PostgreSQL las hace cumplir aunque alguien escriba directamente en la base.

Regla	Mecanismo
El neto siempre es bruto $\times (1 - \%/100)$	monto_neto es una columna generada (no se puede escribir).
Montos positivos y % entre 0 y menos de 100	Restricciones CHECK monto_bruto_positivo, monto_positivo, porcentaje_rango.
Un solo porcentaje vigente por proyecto	Índice único parcial uq_historial_abierto_por_proyecto (WHERE fecha_fin IS NULL) y CHECK vigencia_valida.
Cliente único por empresa	uq_terminal_por_empresa (empresa_id, identificador_terminal).
Cliente sin empresa único por responsable	Índice único parcial uq_terminal_sin_empresa_por_responsable.
El responsable de un cliente con empresa es el de la empresa	Llave foránea compuesta (empresa_id, usuario_id) → empresas(id, usuario_id) con ON UPDATE CASCADE: al reasignar la empresa, sus clientes la siguen.

Regla	Mecanismo
Proyecto único por cliente	uq_proyecto_por_terminal (terminal_id, nombre).
La bitácora no se modifica	Trigger trg_bitacora_inmutable que rechaza UPDATE y DELETE; acciones de un catálogo cerrado (CHECK).

4.4 Migraciones

Versión	Contenido
0001 · Esquema inicial	Usuarios, empresas, terminales con %, movimientos, salidas, bitácora y su trigger.
0002 · Proyectos y métodos de pago	Proyectos con historial de %, métodos de pago, factura y salidas por proyecto (reinicio de datos acordado).
0003 · Clientes sin empresa	empresa_id opcional, responsable en el cliente, llave compuesta, índice parcial y nuevas acciones de bitácora. Conserva los datos.

5. Reglas de negocio implementadas

5.1 Comisiones y saldos

- El % aplicado a una entrada es el vigente en su proyecto el **día de la entrada**; el periodo es [inicio, fin).
- Al cambiar un % con fecha pasada se cierra el periodo vigente, se abre el nuevo y se **recalculan** las entradas afectadas (queda en bitácora).
- Saldo = Σ netos – Σ salidas; se calcula al momento para proyecto, cliente, empresa y la fila «Sin empresa».
- Las salidas no se bloquean por saldo insuficiente (control paralelo), pero se avisa del saldo negativo.
- Proyectos o clientes inactivos no aceptan capturas nuevas; las fechas futuras se rechazan.

5.2 Acceso a la información

- El administrador ve todo. El contador ve las empresas de las que es responsable y los clientes cuyo usuario_id es el suyo (incluye sus clientes sin empresa).
- Un recurso ajeno responde **404** (no 403), para no revelar su existencia.
- Correcciones y eliminaciones exigen motivo y guardan el antes/después en la bitácora.

5.3 Cambio de empresa reversible

- POST /terminales/{id}/empresa mueve al cliente con todo su historial; guarda en bitácora la empresa y el responsable antes y después y el **saldo movido**.
- Un contador solo mueve entre sus empresas o a «Sin empresa»; el administrador a cualquiera (el cliente toma el responsable de la empresa destino).
- POST /terminales/{id}/revertir-movimiento deshace **solo el último** cambio y únicamente si el cliente sigue donde lo dejó; si no, responde 409.
- POST /terminales/{id}/reasignar (admin) cambia el responsable de un cliente sin empresa.

6. API REST

Todos los endpoints viven bajo `/api` y, salvo el inicio de sesión y `/api/health`, requieren el encabezado `Authorization: Bearer <token>`. Los listados son paginados. La documentación interactiva (OpenAPI) se habilita con `DOCS_ENABLED=true` (apagada en producción).

Recurso	Endpoints principales
<code>/auth</code>	<code>POST /login</code> · <code>GET /me</code> · <code>POST /cambiar-password</code>
<code>/usuarios (admin)</code>	<code>GET, POST</code> · <code>GET, PATCH /{id}</code> · <code>POST /{id}/activar</code> · <code>/desactivar</code> · <code>/restablecer-password</code>
<code>/empresas</code>	<code>GET, POST</code> · <code>GET, PATCH /{id}</code> · <code>POST /{id}/reasignar</code> · <code>GET /{id}/saldo</code>
<code>/terminales (clientes)</code>	<code>GET, POST</code> · <code>GET, PATCH /{id}</code> · <code>POST /{id}/activar</code> · <code>/desactivar</code> · <code>GET /{id}/saldo</code> · <code>POST /{id}/empresa</code> · <code>GET /{id}/historial-empresa</code> · <code>POST /{id}/revertir-movimiento</code> · <code>POST /{id}/reasignar</code>
<code>/proyectos</code>	<code>GET, POST</code> · <code>GET, PATCH /{id}</code> · <code>POST /{id}/activar</code> · <code>/desactivar</code> · <code>GET, POST /{id}/porcentajes</code> · <code>GET /{id}/porcentaje-vigente</code> · <code>GET /{id}/saldo</code>
<code>/movimientos (entradas)</code>	<code>GET, POST</code> · <code>GET, PATCH, DELETE /{id}</code>
<code>/salidas</code>	<code>GET, POST</code> · <code>GET, PATCH, DELETE /{id}</code>
<code>/metodos-pago</code>	<code>GET</code> · <code>POST, PATCH /{id}</code> (admin)
<code>/reportes</code>	<code>GET /constructor</code> · <code>/saldos</code> · <code>/estado-cuenta</code> · <code>/conciliacion-diaria</code> · <code>/resumen-mensual</code> · <code>/auditoria-captura</code> · <code>/bitacora</code> · <code>/cierre-diario</code> · <code>POST /cierre-diario/enviar</code>

Los reportes aceptan `formato=json|csv|xlsx|pdf` y los filtros `empresa_id` o `sin_empresa=true`. Los errores se devuelven como JSON con un mensaje en español: 400/422 (validación o regla de negocio), 401 (sesión), 403 (permiso), 404 (no existe o no es suyo), 409 (conflicto) y 429 (demasiados intentos).

7. Seguridad

Control	Implementación
Autenticación	JWT HS256 firmado con SECRET_KEY (mín. 32 caracteres; Render la genera); expira a los 60 minutos. Un usuario desactivado pierde el acceso en la siguiente petición.
Contraseñas	bcrypt; mínimo 10 caracteres con letras y números. Nunca se guardan ni se registran en claro.
Fuerza bruta	slowapi: 5 intentos de acceso por minuto por IP (LOGIN_RATE_LIMIT), con IP real detrás del proxy de Render.
Autorización	Funciones solo_propias y clientes_propios aplicadas en cada consulta; 404 para recursos ajenos.
Validación	Pydantic v2 en todas las entradas; restricciones en la base de datos como segunda barrera.
Auditoría	Bitácora inmutable por trigger.
Contenedor	Se ejecuta con un usuario sin privilegios (uid 1000); documentación OpenAPI desactivada en producción.

8. Calidad: pruebas e integración continua

8.1 Pruebas automáticas

Las pruebas usan pytest con el cliente HTTP de FastAPI contra una base PostgreSQL real, de modo que también se verifican las restricciones y triggers.

Archivo	Pruebas	Qué cubre
test_auth.py	8	Inicio de sesión, expiración, usuarios desactivados, cambio y restablecimiento de contraseña.
test_negocio.py	13	Comisiones por vigencia, recálculo, entradas, salidas, correcciones, métodos de pago, factura.
test_acceso.py	4	Aislamiento entre contadores y permisos de administrador.
test_reportes.py	6	Constructor, saldos, estado de cuenta, conciliación y exportaciones.
test_clientes_sin_empresa.py	10	Alta sin empresa, mover en ambos sentidos, revertir, reasignar, reportes con «Sin empresa».
Total	41	

8.2 Integración continua (GitHub Actions)

Cada `git push` ejecuta `.github/workflows/ci.yml`:

- **Backend:** levanta PostgreSQL 16, aplica `alembic upgrade head`, comprueba con `alembic check` que los modelos y las migraciones coinciden y corre las 41 pruebas.
- **Frontend:** `npm ci`, verificación de tipos y compilación de producción.
- **Docker:** compila la imagen completa.

Render tiene `autoDeployTrigger: checksPass: solo despliega si todo lo anterior pasa`. Dependabot abre periódicamente solicitudes de actualización que pasan por la misma verificación.

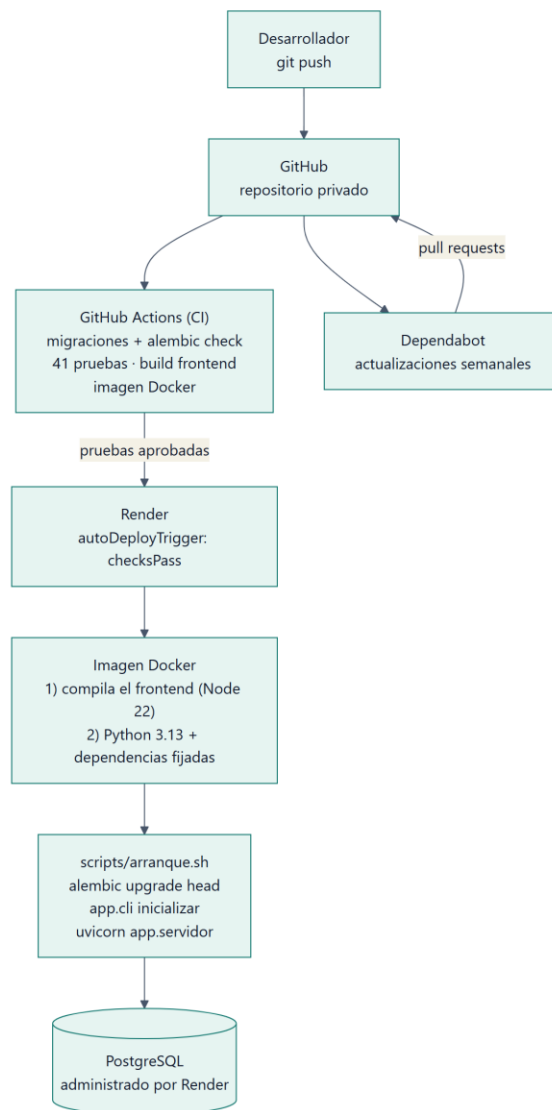


Figura 3. Flujo de integración y despliegue.

9. Despliegue y operación

9.1 Producción (Render)

render.yaml define el servicio web Docker y la base PostgreSQL. Al arrancar, el contenedor ejecuta scripts/arranque.sh:

```
alembic upgrade head          # aplica migraciones pendientes
python -m app.seed            # solo si SEED_DEMO=true
python -m app.cli inicializar  # crea el primer administrador si no hay usuarios
uvicorn app.servidor:servidor --host 0.0.0.0 --port $PORT --proxy-headers
```

9.2 Variables de entorno

Variable	Descripción
DATABASE_URL	Conexión a PostgreSQL (Render la inyecta desde la base).
SECRET_KEY	Clave de firma de los JWT (Render la genera).
ENVIRONMENT / DOCS_ENABLED	production y false en Render.
TIMEZONE	America/Tijuana: define «hoy» para fechas y cierre.
ADMIN_CORREO / ADMIN_PASSWORD / ADMIN_NOMBRE / ADMIN_APELLIDOS	Primer administrador; ADMIN_RESTABLECER=true permite restablecer su contraseña.
SEED_DEMO	true carga datos de demostración.
SCHEDULER_ENABLED, CIERRE_HORA, CIERRE_MINUTO	Cierre diario automático (apagado en el plan gratuito, que se duerme).
SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD, SMTP_FROM, SMTP_SECURITY	Correo para el cierre diario.
CORS_ORIGINS, LOGIN_RATE_LIMIT, ACCESS_TOKEN_EXPIRE_MINUTES	Ajustes opcionales de seguridad.

9.3 Entorno local de desarrollo

```
# Backend (carpeta backend/)
python -m venv .venv
.venv\Scripts\pip install -r requirements-dev.txt
.venv\Scripts\alembic upgrade head
.venv\Scripts\python -m app.seed          # datos de prueba (opcional)
.venv\Scripts\python -m uvicorn app.main:app --port 8000

# Frontend (carpeta frontend/)
npm ci
npm run dev                                # http://localhost:5173, proxy /api → :8000

# Pruebas
.venv\Scripts\python -m pytest
```

10. Limitaciones y trabajo futuro

Tema	Situación / recomendación
Plan gratuito de Render	El servicio se duerme tras 15 min sin uso (~1 min en despertar) y la base gratuita expira a los 30 días con borrado de datos. Para uso real: plan de pago o servidor propio.
RespalDOS	Programar pg_dump diario fuera del servidor y probar la restauración periódicamente.
Cierre diario	Requiere un servicio siempre encendido y SMTP configurado; en el plan gratuito está apagado.
Pruebas del frontend	Agregar pruebas de componentes (Vitest) y de punta a punta (Playwright) a la integración continua.
Factura	Hoy es una marca por entrada usada en filtros y reportes; a futuro puede ligarse al timbrado (CFDI).
Observabilidad	Agregar registro centralizado de errores y monitoreo de disponibilidad.
Mantenimiento	Revisar y aceptar las solicitudes de Dependabot cada semana; actualizar Python/Node en sus versiones con soporte.